Rotation Equivariant Shape Completion using Vector Neuron

Barry Shichen Hu
Technical University of Munich
shichen.hu@tum.de

Abstract

In this project, we perform the task of point cloud completion with a rotation equivariant approach. Existing models are trained under canonical poses and learn on the Cartesian coordinates under canonical poses. As a result, they show poor performance when tested under random rotation. To avoid training with an exhaustive rotation augmentation, we propose a Vector Neuron-based point cloud completion pipeline that recovers a dense complete point cloud from a partial input. The pipeline shows a comparable performance compared to the baseline trained with rotation and outperforms a previous state-of-the-art network.

1. Introduction

Real-world 3d data is usually incomplete due to the nature of the scanning sensors and devices. For example, the point sets obtained by the Lidar sensors are coarse and only capture visible surfaces from a viewing point. Several works have been proposed to tackle this incompleteness by using deep neural network to refine detailed surfaces and predict the geometry in the unseen area. Despite achieving extraordinary performance, these networks have one obvious drawback in common, the networks does not achieve rotation equivariance. In other word, if the input point cloud is randomly rotated, the change of the cartesian coordinates of the input would cause the network to dysfunction. Ideally, the completion task should not be affected by the rigid body transformation especially the rotation of the input.

Inspired by the invention of Vector Neuron framework(VN) [2] and the development of recent completion methods, we propose a model that achieves rotation equivariance through a two stages reconstruction. First a global shape is predicted in the form of a coarse point cloud that covers the overall surface of the object, then a Folding-based decoder [10] is utilized to densify the coarse point cloud and refine the local geometry. The resulting pipeline is an end-to-end trainable model that achieves comparable performance with the baseline.

The contribution of this project is three-fold:

1. We propose a rotation equivariant pipeline that achieves a comparable performance to the baseline when tested with random rotation.
2. We conducted various ablation studies and summarized the design choices that leads to performance gain.
3. We discuss the pros and cons of VN, which places should be modified using VN in a completion pipeline. In addition, we discuss why IoU and Chamfer Distance might give different evaluation results.

2. Related Work

Point Cloud Completion Various models use the "two-stage completion" strategy to solve the completion task. Yuan et al. [12] first uses a PointNet module [5] to extract the input features and generates a coarse point cloud. Then, a Folding-based operation is deployed to generate the dense point cloud based on the coarse points. Yu et al. [11] first uses a DGCNN [8] and a transformer encoder to generate a coarse point cloud, then uses the predicted coarse points as query points and utilize a transformer decoder followed by a FoldingNet to approximate the local geometry. [9] follows a different workflow, it uses a 3D grid as an intermediate representation to refine the captured features and produce a dense point cloud directly. Regardless of the architectural differences, all of the available completion methods depend on the absolute coordinates of the input, thus do not achieve rotation equivariance unless being explicitly trained with rotational augmentation.

Rotation Invariant Methods Many rotation invariant neural network architectures are built for classification and segmentation tasks and they utilize SO(3) representations. Zhang et al. [13] uses geometric cues like distances and angles around a centroid and a reference point to build rotation invariant features. Kim et al. [4] extracts local features by local reference frame and PCA, then hierarchically aggregate information in a similar fashion as the local feature extraction.

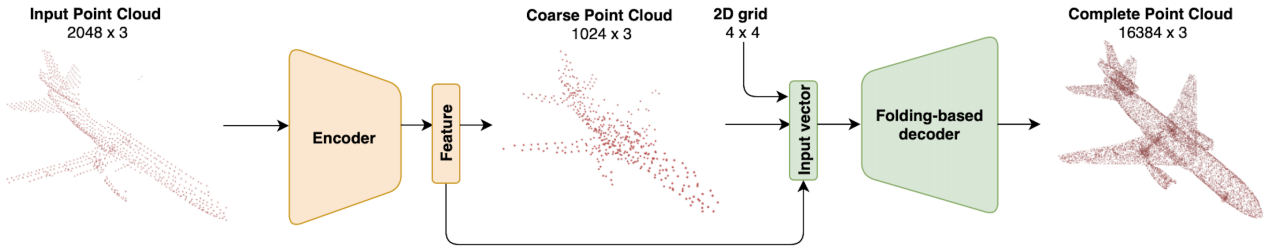


Figure 1. The overall architecture of the pipeline that is used in this project.

Rotation Equivariant Methods To avoid training with an exhaustive rotation augmentation, rotation equivariant methods have emerged. [6] uses convolution filters built from spherical harmonics to achieve rotation equivariance. [3] extracts rotation invariant features in local scale and applies attention upon them in local neighborhood. While guaranteeing equivariance by construction, it is hard to incorporate these methods into existing pipelines. Vector Neuron(VN), proposed by Deng et al. [2], is a general SO(3) framework that is easy to adapt. VN framework is built on the associativity of matrix multiplication, and it includes Vector Neuron version of the linear, activation and normalization layers.

3. Method

3.1. Overview

The pipeline proposed in this project consists of two stages: 1) coarse point cloud prediction and 2) dense point cloud production. Figure 1 shows the architecture overview. The input partial point cloud is passed into an encoder for feature aggregation and a decoder is used to produce the final dense point cloud. The pipeline is implemented with Vector Neuron.

3.2. Vector Neuron

Vector Neuron framework provides a set of novel network layers that are rotation equivariant by design. In a normal deep learning network, the input and output of each layer are vectors of scalars. By contrast, the input and output of each vector neuron layer are vectors of three-element tuples, in analogous to the x,y,z coordinates of 3d points. As a results, Vector Neuron enables the preservation of the rotation based on the associativity of matrix operations.

Analogous to common deep learning layers, Vector Neuron framework provides four layers, named as VNLinear, VNLeakyReLU, VNBatchNorm and VNLayerNorm which correspond to Linear, LeakyReLU, BatchNorm and LayerNorm in a normal deep learning architecture. Since each VN layer preserve the rotation, the network built completely with the Vector Neuron layers are rotation equivariant by design.

3.3. Encoder - Coarse Point Cloud Prediction

We investigate three different pipelines for coarse point cloud prediction, as illustrated in Figure 2. All of these three pipelines are implemented with Vector Neuron layers. The first pipeline is adopted from the baseline, PoinTr network [11]. It first uses a DGCNN model [8] to down-sample and aggregate input point cloud. Then, the resulting features are passed into a transformer encoder [7] to model the relationship among them. Furthermore, a max-pooling operation is used to generate a global feature vector, based on which a MLP network is applied to produce the coarse points.

Pipeline b and c follow the same structure as pipeline a, with only a single DGCNN or PointNet for extracting input features. For all three pipelines, a coarse point cloud and a global vector are produced and passed to the second stage.

3.4. Decoder - Dense Point Cloud Prediction

For the second stage, we adopted the Folding operation from [10]. In a nutshell, the Folding operation places a 2d grid of points (also called Folding seed in [10] and [12]) at each coarse point location and deform the 2d grids into the local geometry. For each coarse point, we append the global feature vector from the encoder, the coordinates of the coarse point itself and the coordinates of the points in the 2d grid. The resulting vector is then passed into the FoldingNet to predict the relative position for each points in the deformed 2d grid.

3.5. Loss Function

We uses two metrics for evaluation, Chamfer Distance (L1) and IoU. Chamfer Distance measures how close two point sets are, we define Chamfer Distance loss (L1) between two point sets S_1, S_2 as:

$$CD(S_1, S_2) = \frac{1}{|S_1|} \sum_{x \in S_1} \min_{y \in S_2} ||x - y|| + \frac{1}{|S_2|} \sum_{x \in S_2} \min_{y \in S_1} ||x - y|| \quad (1)$$

IoU measures how accurate the point cloud predictions are. To compute IoU, we first generate a voxel mesh from

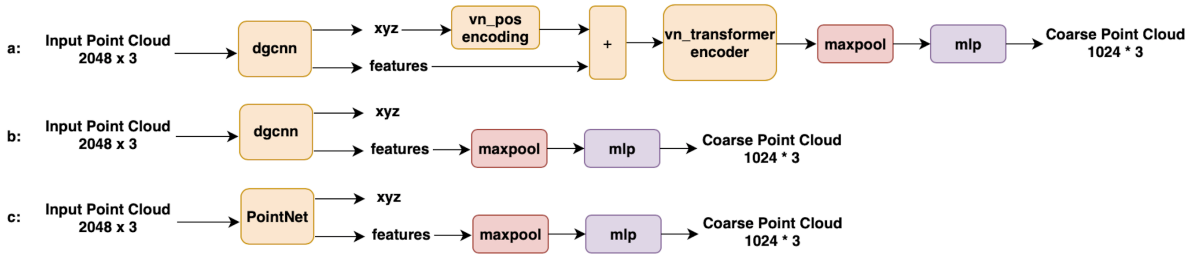


Figure 2. Models for coarse points production.

a predicted point cloud by segmenting the occupied space into a 64 x 64 x 64 grid. Then the occupancy of each grid cell is determined by the existence of any predicted point inside the grid cell. After obtaining the voxel grids for the prediction and the ground truth, we can compute the Intersection over Union.

4. Experiments

4.1. Datasets and Implementation Details

Datasets. We use the same dataset as in [12] and [11], which is a subset of ShapeNet dataset [1]. The dataset consists of 30974 shapes, with 28974 shapes in training set, 800 shapes in val set and 1200 shapes in test set. Each input point cloud is a subset of the corredponding ground truth point cloud and the inputs are single view partial point clouds. In addition, we use the method described in Section 3.5 to generate the ground truth voxel grids.

Implementation Details. The experiments were run with the PyTorch framework on an NVIDIA 3080Ti GPU. For coarse point prediction, we used a batch size of 24, learning rate of 1e-3 and the models were trained for 300 epochs. For dense point prediction without end-to-end training, we kept the pre-trained encoder frozen, with a batch size of 6, learning rate of 1e-3, and we trained the models for 150 epochs. For end-to-end dense point prediction training, we used a batch size of 3 instead. For both coarse and dense point cloud prediction, a learning rate scheduler is applied to decay the learning rate with a factor of 0.8 for every 50 epochs and AdamW is used as the optimizer. In addition, we use gradient clipping at an l_2 norm of 0.5.

4.2. Quantitative Results

In this section, we show the performance of coarse and dense point cloud prediction for the pipelines proposed in Section 3.

Coarse point prediction We showed the evaluation for coarse point prediction measured in Chamfer Distance (L1) under the Table 1. Notably, the baseline trained with rotation produces the best coarse point sets. PointNet implemented with Vector Neuron framework, denoted as VN-PointNet, and baseline modified with Vector Neuron framework, denoted as VNBaseline, produce better results than the rest models.

Model	CDL1 (448 points)	CDL1 (1024 points)
DGCNN (so3)	20.88	20.48
VNDGCNN	23.53	24.09
VNPointNet	20.94	18.79
VNbaseline	20.5	19.77
baseline (so3)	**15.83**	**16.83**

Table 1. Coarse point cloud prediction measured with random rotation, the prefix VN means that the model is implemented with Vector Neuron. VN models are trained under canonical poses.

Model	CDL1	IoU (64^3)
VNPointNet_1024 + VNFoldingNet	12.32	**19.90%**
PCN(so3)	14.95	17.3%
baseline(so3)	**10.23**	13.93%

Table 2. Dense point cloud prediction measured with random rotation. The VN pipeline is trained under canonical poses.

Dense point prediction The results for dense point cloud prediction is in Table 2, we measured both L1 Chamfer Distance and IoU. The best Vector Neuron configuration, **VNPointNet_1024+VNFoldingNet** (aka, PointNet implemented with Vector Neuron as the encoder to produce 1024 coarse points and FoldingNet implemented with Vector Neuron as the decoder) is able to to achieve a Chamfer Distance of 12.32 on the test set, this is better than Point Completion Network (PCN) trained with rotation, however, not as good as the baseline trained with rotation.

4.3. Qualitative results

The visualization of our method is in Figure 3. Our model is able to capture rotation-equivariant features and predict the complete shape regardless of the rotation. However, we also notice that the local reconstruction is noisy.

VNTransformer Configuration	CDL1
num attention head 12, qkv dim as 384	**20.49**
num attention head 8 , qkv dim as 256	21.02
num attention head 4 , qkv dim as 128	21.59

Table 3. Coarse Point Prediction by VNTransformer, qkv means query, key and value

Model	configuration	CDL1
VNDGCNN	Knn in xyz space	24.03
	Knn in feature space	**23.35**
DGCNN (SO3)	with fps	**21.22**
	without fps	22.35

Table 4. Coarse Point Prediction with (VN)DGCNN

4.4. Ablation Studies

In this section, we discuss the influence of various design choices on the models being investigated.

Vector Neuron Transformer We investigated the effect of different position encodings for pipeline a in Figure 2. We witnessed that without a position encoding, the model does not overfit well. In addition, simply using the xyz coordinates (repeated to meet the feature dimension of the input features) produces better performance than position encodings obtained from applying a vector neuron layer upon the xyz coordinates.

We further investigated how to increase the size of VNTransformer to achieve better results. We discovered that increasing the dimension of query, key and value together with the number of attention heads leads to better results, while increasing one of depth, dimension and attention head alone does not provides a performance gain. The experiments results are in Table 3.

DGCNN We investigated various ways of aggregating features in DGCNN and found that applying downsampling by farthest point sampling and finding knns in the feature space gives the best results. These are recorded in Table 4

Folding Seed We first modified the Folding decoder using Vector Neuron layers and run the experiment without the Folding seed (2d grid of points). This leads to a high overfitting loss. Thus, the further experiments were conducted with the 2D grid. Even though a decent quantitative result on dense prediction was witnessed in Section 4.2, we observed that the visualization in Section 4.3 is noisy on the local scale. Therefore, we furthur tested the trained model with the 2D grid being rotated by the ground truth rotation during testing. We witnessed that applying the ground truth rotation to the 2D grid does not provide a significant better results. Thus we hypothesize that the current setup of FoldingNet does not learn the local geometry well and produces a somewhat noisy reconstruction around each coarse point.

Encoder	Dense CDL1
DGCNN_so3_448 (frozen)	22.37
DGCNN_so3_1024 (frozen)	16.01
VN_PointNet_448 (frozen)	12.66
VN_PointNet_1024 (frozen)	**12.32**
VN_PointNet_1024 (end-to-end)	12.54
baseline_so3_1024 (frozen)	14.06

Table 5. Dense Point Prediction with various encoders. VNFoldingNet is used as the decoder here.

Encoder Choices We investigated various choices of encoders in the pipeline and found that the best configuration uses a VNpointNet as the encoder. See results in Table 5.

5. Discussion

Pros and cons of Vector Neuron Vector Neuron layers are analogous to the common neuron network layers, thus, it can be easily applied to existing pipelines. However, the Vector Neuron framework has its obvious drawbacks. First, it is very memory consuming. During our experiments, pipelines modified with Vector Neuron consumes more than three times more memory than the pipeline before modification, even though the size of VN pipelines are smaller. Furthermore, simply increasing the capacity of the VN pipelines does not necessarily increase the model's learning capacity during training. Thus, the best practices of Vector Neuron is yet to be discovered.

Places to apply Vector Neuron in a pipeline From Table 5, the pipeline that uses Baseline_so3_1024 as the encoder produces poor dense point clouds even though this encoder produces the best coarse points (in Table 1). Furthermore, when using DGCNN_so3_1024 as the encoder, the dense prediction is significantly worse than most of the other encoders listed in Table 1, even though its coarse prediction is not much worse than other encoders. Thus, we hypothesize that VNFoldingNet cannot utilize the features learnt by training a non-VN encoder with rotation.

In addtion, we experimented using VNPointNet_1024 as the encoder and normal FoldingNet as the decoder, we obtained a Chamfer Distance of 15.05 when evaluated with rotation. This is significantly higher than that when using a VNFoldingNet. Moreover, a non-trivial performance gap is observed between canonical poses and random rotation. Thus, We hypothesize that A normal FoldingNet cannot use the features learnt from VN encoders to achieve rotation equivariance. In conclusion, we hypothesize that Vector Neuron should be applied to the whole pipeline.

Why Chamfer Distance and IoU give different results

From Table 2, we can see that the baseline(so3) produces a better result in Chamfer Distance, however, the VN pipeline is better when measured with IoU. We figures that two reasons might be behind this: First, the point cloud returned by the VN pipeline is indeed able to recover the overall surface of the shape, however, the point cloud is more unevenly distributed than the point clouds returned by the baseline(so3). Density is not taken into account in IoU calculation while it is reflected in Chamfer Distance. Second, the point cloud produced by the VN pipeline is more noisy on the local scale, since we voxelize the point clouds before calculating the IoU, this local noisiness is ignored in the IoU calculation. These two reasons highlight the necessity of tackling density issue and local noisy reconstruction if we aim to further improve the VN pipeline.

6. Conclusion

With Vector Neuron framework, We have built a rotation equivariant point cloud completion pipeline that have comparable performance as the baseline. However, there is still plenty of room for further development. For example, transformer can be applied in the decoder; density issue needs to be tackled; and to achieve fully rotation equivariant, we can deform a sphere instead of a 2D grid. We are excited to continue working in these directions.

References

[1] Angel X. Chang, Thomas A. Funkhouser, Leonidas J. Guibas, Pat Hanrahan, Qi-Xing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. Shapenet: An information-rich 3d model repository. *CoRR*, abs/1512.03012, 2015. URL http://arxiv.org/abs/1512.03012. 3

[2] Congyue Deng, Or Litany, Yueqi Duan, Adrien Poulenard, Andrea Tagliasacchi, and Leonidas J. Guibas. Vector neurons: A general framework for so(3)-equivariant networks. *CoRR*, abs/2104.12229, 2021. URL https://arxiv.org/abs/2104.12229. 1, 2

[3] Fabian B. Fuchs, Daniel E. Worrall, Volker Fischer, and Max Welling. Se(3)-transformers: 3d roto-translation equivariant attention networks. *CoRR*, abs/2006.10503, 2020. URL https://arxiv.org/abs/2006.10503. 2

[4] Seohyun Kim, Jaeyoo Park, and Bohyung Han. Rotation-invariant local-to-global representation learning for 3d point cloud. *CoRR*, abs/2010.03318, 2020. URL https://arxiv.org/abs/2010.03318. 1

[5] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space, 2017. 1

[6] Nathaniel Thomas, Tess E. Smidt, Steven Kearnes, Lusann Yang, Li Li, Kai Kohlhoff, and Patrick Riley. Tensor field networks: Rotation- and translation-equivariant neural networks for 3d point clouds. *CoRR*, abs/1802.08219, 2018. URL http://arxiv.org/abs/1802.08219. 2

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL http://arxiv.org/abs/1706.03762. 2

[8] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. Dynamic graph CNN for learning on point clouds. *CoRR*, abs/1801.07829, 2018. URL http://arxiv.org/abs/1801.07829. 1, 2

[9] Haozhe Xie, Hongxun Yao, Shangchen Zhou, Jiageng Mao, Shengping Zhang, and Wenxiu Sun. Grnet: Gridding residual network for dense point cloud completion. *CoRR*, abs/2006.03761, 2020. URL https://arxiv.org/abs/2006.03761. 1

[10] Yaoqing Yang, Chen Feng, Yiru Shen, and Dong Tian. Foldingnet: Interpretable unsupervised learning on 3d point clouds. *CoRR*, abs/1712.07262, 2017. URL http://arxiv.org/abs/1712.07262. 1, 2

[11] Xumin Yu, Yongming Rao, Ziyi Wang, Zuyan Liu, Jiwen Lu, and Jie Zhou. Pointr: Diverse point cloud completion with geometry-aware transformers. *CoRR*, abs/2108.08839, 2021. URL https://arxiv.org/abs/2108.08839. 1, 2, 3

[12] Wentao Yuan, Tejas Khot, David Held, Christoph Mertz, and Martial Hebert. PCN: point completion network. *CoRR*, abs/1808.00671, 2018. URL http://arxiv.org/abs/1808.00671. 1, 2, 3

[13] Zhiyuan Zhang, Binh-Son Hua, David W. Rosen, and Sai-Kit Yeung. Rotation invariant convolutions for 3d point clouds deep learning. *CoRR*, abs/1908.06297, 2019. URL http://arxiv.org/abs/1908.06297. 1

Supplementary Material

A. Additional Visualization

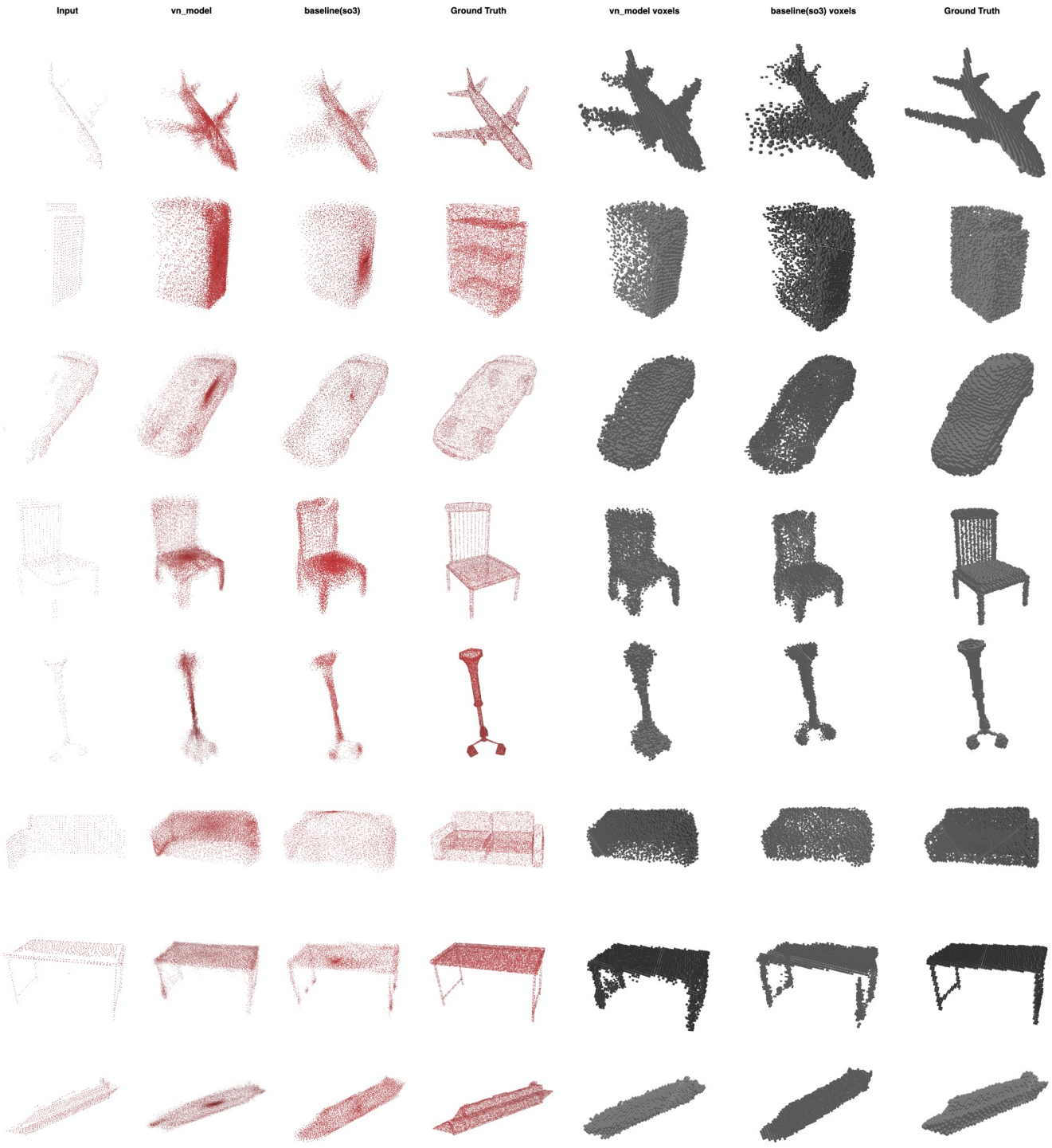


Figure 3. Some predictions from our Vector Neuron Pipeline.